

DECISION MAKING AND BRANCHING

5.1 INTRODUCTION

We have seen that a C program is a set of statements which are normally executed sequentially in the order in which they appear. This happens when no options or no repetitions of certain calculations are necessary. However, in practice, we have a number of situations where we may have to change the order of execution of statements based on certain conditions, or repeat a group of statements until certain specified conditions are met. This involves a kind of decision making to see whether a particular condition has occurred or not and then direct the computer to execute certain statements accordingly.

C language possesses such decision making capabilities and supports the following statements known as *control* or *decision making* statements.

1. **if** statement
2. **switch** statement
3. Conditional operator statement
4. **goto** statement

We have already used some of these statements in the earlier examples. Here we shall discuss their features, capabilities and applications in more detail.

5.2 DECISION MAKING WITH IF STATEMENT

The **if** statement is a powerful decision making statement and is used to control the flow of execution of statements. It is basically a *two-way* decision statement and is used in conjunction with an expression. It takes the following form:

if (*test expression*)

It allows the computer to evaluate the expression first and then, depending on whether the value of the expression (relation or condition) is 'true' (non-zero) or 'false' (zero), it

transfers the control to a particular statement. This point of program has two *paths* to follow, one for the true condition and the other for the false condition as shown in Fig. 5.1.

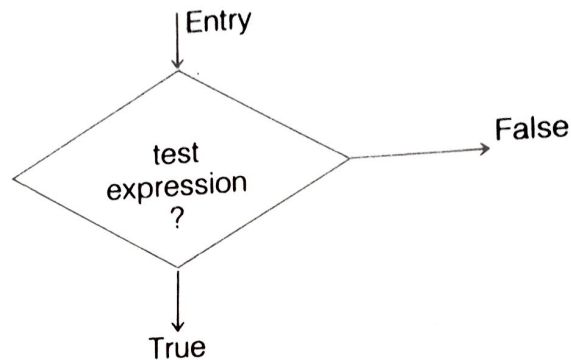


Fig. 5.1 Two-way branching

Some examples of decision making, using **if** statement are:

1. **if** (bank balance is zero)
 borrow money
2. **if** (room is dark)
 put on lights
3. **if** (code is 1)
 person is male
4. **if** (age is more than 55)
 person is retired

The **if** statement may be implemented in different forms depending on the complexity of conditions to be tested.

1. Simple **if** statement
2. **if....else** statement
3. Nested **if....else** statement
4. **else if** ladder.

5.3 SIMPLE IF STATEMENT

The general form of a simple **if** statement is

```

if(test expression)
{
    statement-block;
}
statement-x;
  
```

The 'statement-block' may be a single statement or a group of statements. If the *test expression* is true, the *statement-block* will be executed; otherwise the *statement-block* will be skipped and the execution will jump to the *statement-x*. Remember, when the condition is true both the *statement-block* and the *statement-x* are executed in sequence. This is illustrated in Fig. 5.2.

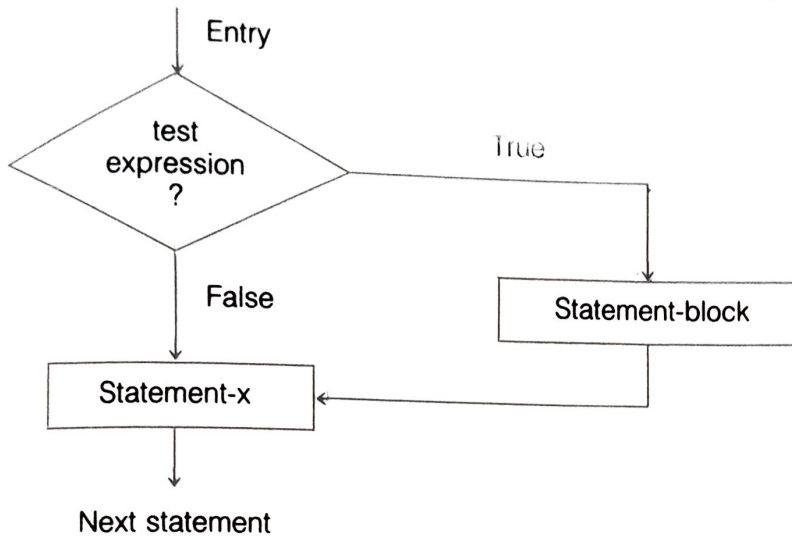


Fig. 5.2 Flowchart of simple *If* control

Consider the following segment of a program that is written for processing of marks obtained in an entrance examination.

```

.....
.....
if (category == SPORTS)
{
    marks = marks + bonus_marks;
}
printf("%f", marks);
.....
.....
  
```

The program tests the type of category of the student. If the student belongs to the SPORTS category, then additional `bonus_marks` are added to his marks before they are printed. For others, `bonus_marks` are not added.

Example 5.1

The program in Fig. 5.3 reads four values `a`, `b`, `c`, and `d` from the terminal and evaluates the ratio of $(a+b)$ to $(c-d)$ and prints the result, if $c-d$ is not equal to zero.

The program given in Fig. 5.3 has been run for two sets of data to see that the paths function properly. The result of the first run is printed as

Ratio = -3.181818

The second run has neither produced any results nor any message. During the second run, the value of $(c-d)$ is equal to zero and therefore the statements contained in the statement-block are skipped. Since no other statement follows the statement-block, program stops without producing any output.

Note the use of **float** conversion in the statement evaluating the **ratio**. This is necessary to avoid truncation due to integer division.

Program

```

/*****
/***** ILLUSTRATION OF if STATEMENT *****/
/*****/

main( )
{
    int a, b, c, d;
    float ratio;

    printf("Enter four integer values\n");
    scanf("%d %d %d %d", &a, &b, &c, &d);

    if (c-d != 0)
    {
        ratio = (float) (a+b) / (float) (c-d);
        printf("Ratio = %f\n", ratio);
    }
}

```

Output

```

Enter four integer values
12 23 34 45
Ratio = -3.181818

Enter four integer values
12 23 34 34

```

Fig. 5.3 Illustration of simple *if* statement

The simple *if* is often used for counting purposes. The example 5.2 illustrates this.

Example 5.2

The program in Fig. 5.4 counts the number of boys whose weight is less than 50 kgs and height is greater than 170 cm.

The program has to test two conditions, one for weight and another for height. This is done using the compound relation

```
if (weight < 50 && height > 170)
```

This would have been equivalently done using two *if* statements as follows:

```

if (weight < 50)
    if (height > 170)
        count = count + 1;

```

If the value of **weight** is less than 50, then the following statement is executed, which in turn is another *if* statement. This *if* statement tests **height** and if the **height** is greater than 170, then the **count** is incremented by 1.

Program

```

/*****
/*                                COUNTING WITH if                                */
*****/
main( )
{
    int count, i;
    float weight, height;
    count = 0;
    printf("Enter weight and height for 10 boys\n");
    for (i = 1; i <= 10; i++)
    {
        scanf("%f %f", &weight, &height);
        if(weight < 50 && height > 170)
            count = count + 1;
    }
    printf("Number of boys with weight < 50 kgs\n");
    printf("and height > 170 cm = %d\n", count);
}

```

Output

```

Enter weight and height for 10 boys
45 176.5
55 174.2
47 168.0
49 170.7
54 169.0
53 170.5
49 167.0
48 175.0
47 167
51 170
Number of boys with weight < 50 kgs
and height > 170 cm = 3

```

Fig. 5.4 Use of **if** for counting**5.4 THE IF... ELSE STATEMENT**

The if...else statement is an extension of the simple if statement. The general form is

```

if (test expression)
{
    True-block statement(s)
}
else
{
    False-block statement(s)
}
statement-x

```

If the *test expression* is true, then the *true-block statement(s)*, immediately following the *if* statement are executed; otherwise, the *false-block statement(s)* are executed. In either case, either *true-block* or *false-block* will be executed, not both. This is illustrated in Fig. 5.5. In both the cases, the control is transferred subsequently to *statement-x*.

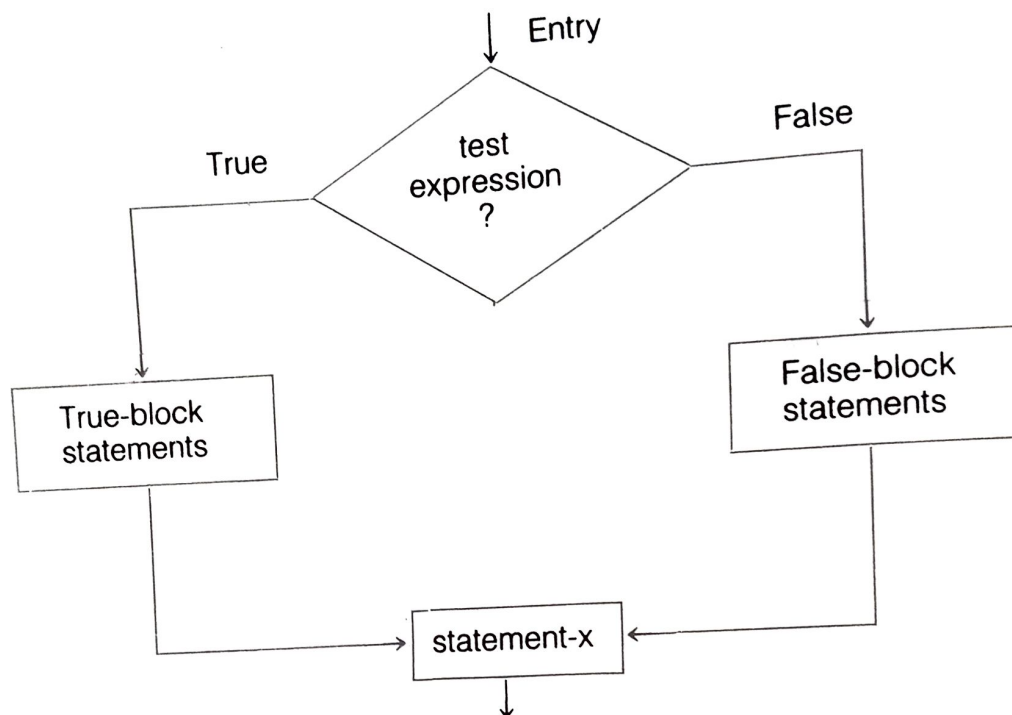


Fig. 5.5 Flowchart of *if.....else* control

Let us consider an example of counting the number of boys and girls in a class. We use code 1 for a boy and 2 for a girl. The program statement to do this may be written as follows:

```

.....
.....
if (code == 1)
    boy = boy + 1;
if (code == 2)
    girl = girl + 1;
.....
.....

```

The first test determines whether or not the student is a boy. If yes, the number of boys is increased by 1 and the program continues to the second test. The second test again determines whether the student is a girl. This is unnecessary. Once a student is identified as a boy, there is no need to test again for a girl. A student can be either a boy or a girl, not both. The above program segment can be modified using the **else** clause as follows:

```

.....
.....
if (code == 1)
    boy = boy + 1;
else
    girl = girl + 1;
XXXXXXXXXX
.....

```

Here, if the code is equal to 1, the statement `boy = boy + 1;` is executed and the control is transferred to the statement `xxxxxx`, after skipping the `else` part. If the code is not equal to 1, the statement `boy = boy + 1;` is skipped and the statement in the `else` part `girl = girl + 1;` is executed before the control reaches the statement `xxxxxxxx`.

Consider the program given in Fig. 5.3. When the value $(c - d)$ is zero, the ratio is not calculated and the program stops without any message. In such cases we may not know whether the program stopped due to a zero value or some other error. This program can be improved by adding the `else` clause as follows:

```

.....
.....
if (c-d != 0)
{
    ratio = (float)(a+b)/(float)(c-d);
    printf("Ratio = %f\n", ratio);
}
else
    printf("c-d is zero\n");
.....
.....

```

Example 5.3

A program to evaluate the power series

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots + \frac{x^n}{n!}, \quad 0 < x < 1$$

is given in Fig. 5.6. It uses `if.....else` to test the accuracy.

The power series contains the recurrence relationship of the type

$$T_n = T_{n-1} \cdot \left(\frac{x}{n}\right) \text{ for } n > 1$$

$$\begin{aligned} T_1 &= x \\ T_0 &= 1 \end{aligned} \quad \text{for } n = 1$$

If T_{n-1} (usually known as *previous term*) is known, then T_n (known as *present term*) can be easily found by multiplying the *previous term* by x/n . Then

$$e^x = T_0 + T_1 + T_2 + \dots + T_n = \text{sum}$$

The program uses `count` to count the number of terms added. The program stops when the value of the term is less than 0.0001 (**ACCURACY**). Note that when a term is less than **ACCURACY**, the value of n is set equal to 999 (a number higher than 100) and therefore the `while` loop terminates. The results are printed outside the `while` loop.

5.5 NESTING OF IF... ELSE STATEMENTS

When a series of decisions are involved, we may have to use more than one `if...else` statement in *nested* form as follows:

Program

```

/*****
 *          EXPERIMENT WITH if...else STATEMENT
 *****/
#define ACCURACY 0.0001

main( )
{
    int n, count;
    float x, term, sum;

    printf("Enter value of x:");
    scanf("%f", &x);

    n = term = sum = count = 1;
    while (n <= 100)
    {
        term = term * x/n;
        sum = sum + term;
        count = count + 1;
        if (term < ACCURACY)
            n = 999;
        else
            n = n + 1;
    }

    printf("Terms = %d Sum = %f\n", count, sum);
}

```

Output

```

Enter value of x:0
Terms = 2 Sum = 1.000000

Enter value of x:0.1
Terms = 5 Sum = 1.105171

Enter value of x:0.5
Terms = 7 Sum = 1.648720

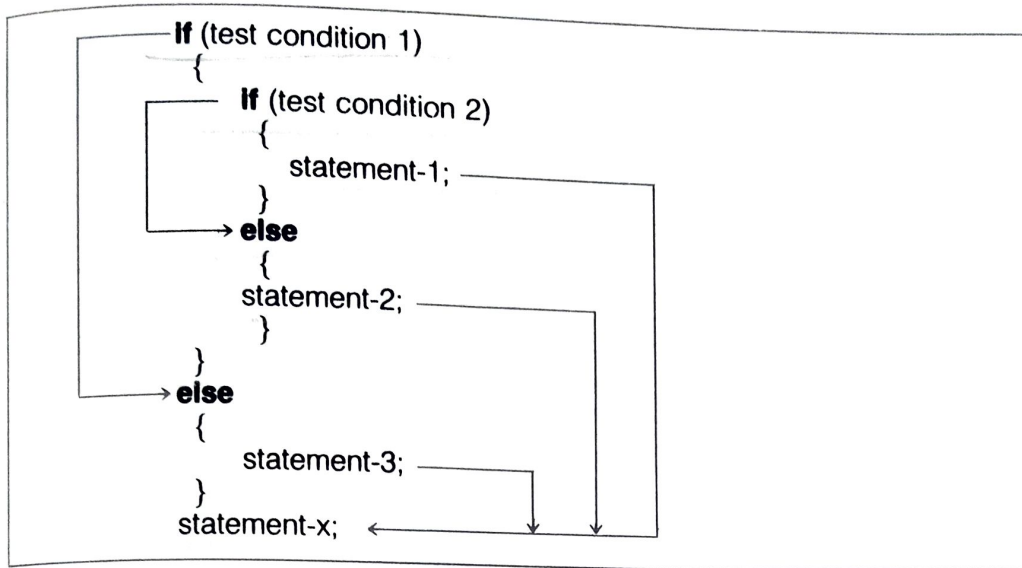
Enter value of x:0.75
Terms = 8 Sum = 2.116997

Enter value of x:0.99
Terms = 9 Sum = 2.691232

Enter value of x:1
Terms = 9 Sum = 2.718279

```

Fig. 5.6 Illustration of **if...else** statement



The logic of execution is illustrated in Fig. 5.7. If the *condition-1* is false, the statement-3 will be executed; otherwise it continues to perform the second test. If the *condition-2* is true, the statement-1 will be evaluated; otherwise the statement-2 will be evaluated and then the control is transferred to the statement-x.

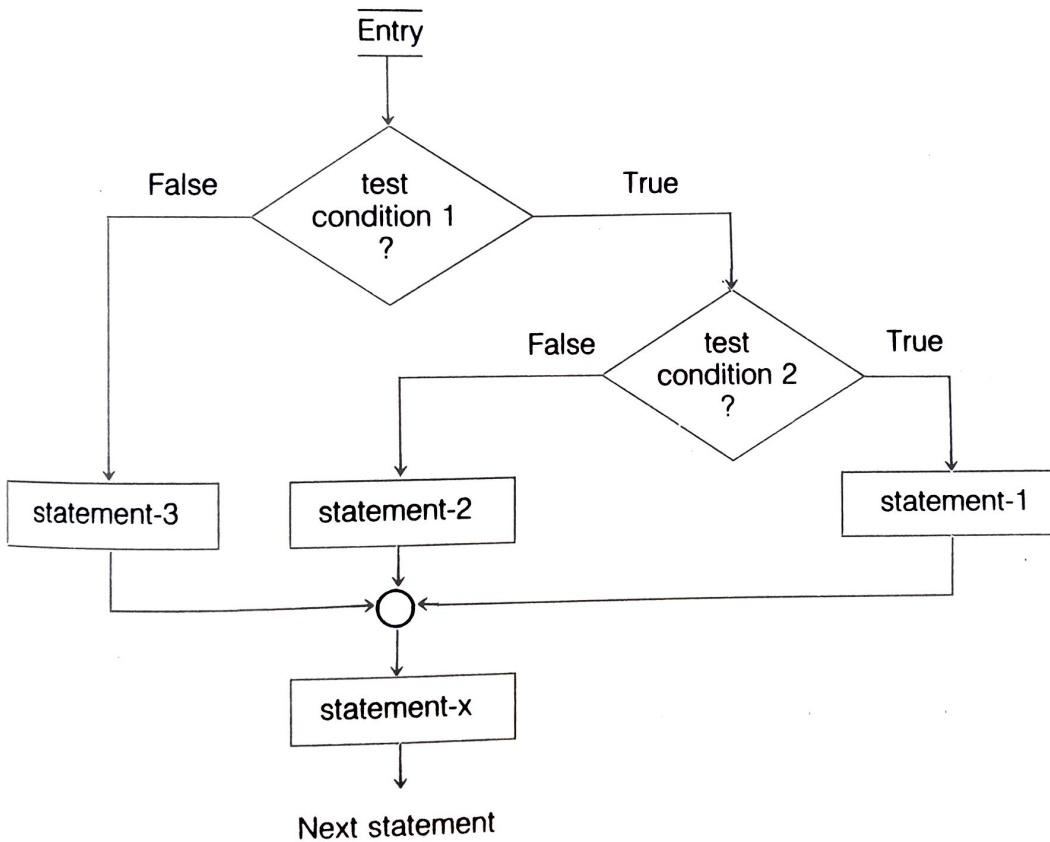


Fig. 5.7 Flowchart of nested **if..else** statements

A commercial bank has introduced an incentive policy of giving bonus to all its deposit holders. The policy is as follows: A bonus of 2 per cent of the *balance* held on 31st December is given to every one, irrespective of their balances, and 5 per cent is given to female account holders if their balance is more than Rs.5000. This logic can be coded as follows:

```

.....
if (sex is female)
{
    if (balance > 5000)
        bonus = 0.05 * balance;
    else
        bonus = 0.02 * balance;
}
else
{
    bonus = 0.02 * balance;
}
balance = balance + bonus;
.....

```

When nesting, care should be exercised to match every **if** with an **else**. Consider the following alternative to the above program (which looks right at the first sight):

```

if (sex is female)
    if (balance > 5000)
        bonus = 0.05 * balance;
    else
        bonus = 0.02 * balance;
balance = balance + bonus;

```

There is an ambiguity as to over which **if** the **else** belongs to. In C, an **else** is linked to the closest non-terminated **if**. Therefore, the **else** is associated with the inner **if** and there is no **else** option for the outer **if**. This means that the computer is trying to execute the statement

balance = balance + bonus;

without really calculating the bonus for the male account holders.

Consider another alternative which also looks correct:

```

if (sex is female)
{
    if (balance > 5000)
        bonus = 0.05 * balance;
}
else
{
    bonus = 0.02 * balance;
    balance = balance + bonus;
}

```

In this case, **else** is associated with the outer **if** and therefore bonus is calculated for the male account holders. However, bonus for the female account holders, whose balance is equal to or less than 5000 is not calculated because of the missing **else** option for the inner **if**.

Example 5.4

The program in Fig. 5.8 selects and prints the largest of the three numbers using nested if...else statements.

Program

```

/*****
/*      SELECTING THE LARGEST OF THREE VALUES      */
*****/

main( )
{
    float A, B, C;

    printf("Enter three values\n");
    scanf("%f %f %f", &A, &B, &C);

    printf("\nLargest value is  ");

    if (A>B)
    {
        if (A>C)
            printf("%f\n", A);
        else
            printf("%f\n", C);
    }
    else
    {
        if (C>B)
            printf("%f\n", C);
        else
            printf("%f\n", B);
    }
}

```

Output

```

Enter three values
23445  67379  88843

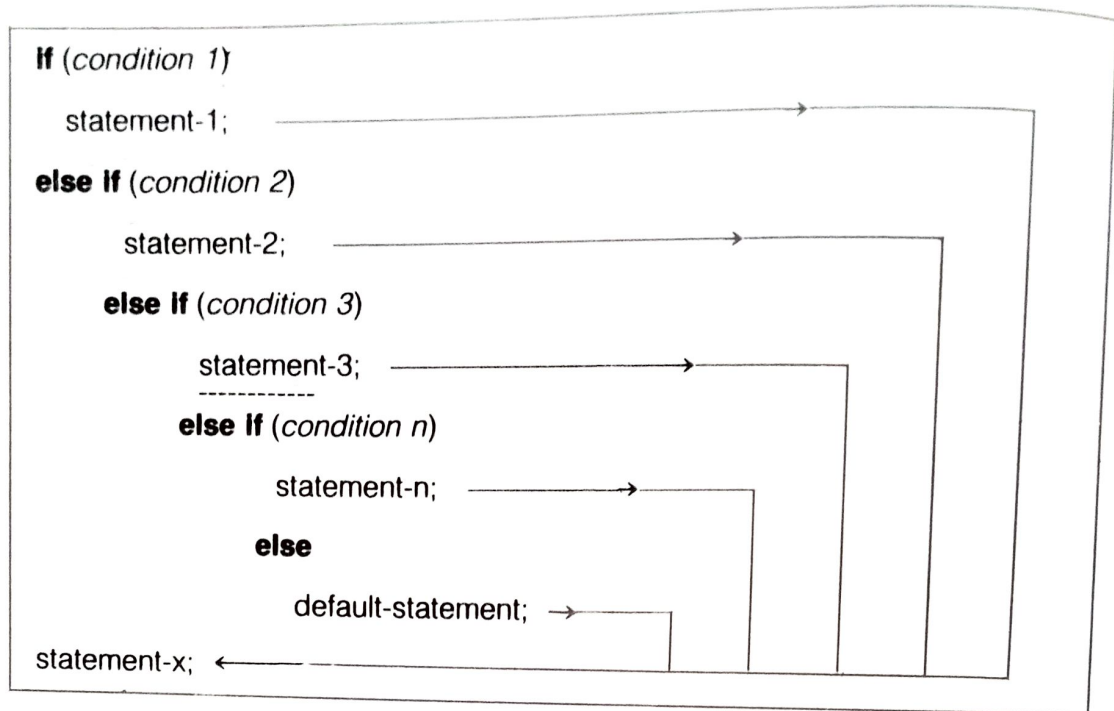
Largest value is  88843.000000

```

Fig. 5.8 *Selecting the largest of three numbers*

5.6 THE ELSE IF LADDER

There is another way of putting ifs together when multipath decisions are involved. A multipath decision is a chain of ifs in which the statement associated with each else is an if. It takes the following general form:



This construct is known as the **else if ladder**. The conditions are evaluated from the top (of the ladder), downwards. As soon as a true condition is found, the statement associated with it is executed and the control is transferred to the *statement-x* (skipping the rest of the ladder). When all the *n* conditions become false, then the final **else** containing the *default-statement* will be executed. Fig.5.9 shows the logic of execution of **else if** ladder statements.

Let us consider an example of grading the students in an academic institution. The grading is done according to the following rules:

Average marks	Grade
80 to 100	Honours
60 to 79	First Division
50 to 59	Second Division
40 to 49	Third Division
0 to 39	Fail

This grading can be done using the **else if** ladder as follows:

```

if (marks > 79)
    grade = "Honours";
else if (marks > 59)
    grade = "First Division";
else if (marks > 49)
    grade = "Second Division";
else if (marks > 39)
    grade = "Third Division";
else
    grade = "Fail";
printf ("%s\n", grade);
  
```

Consider another example given below:

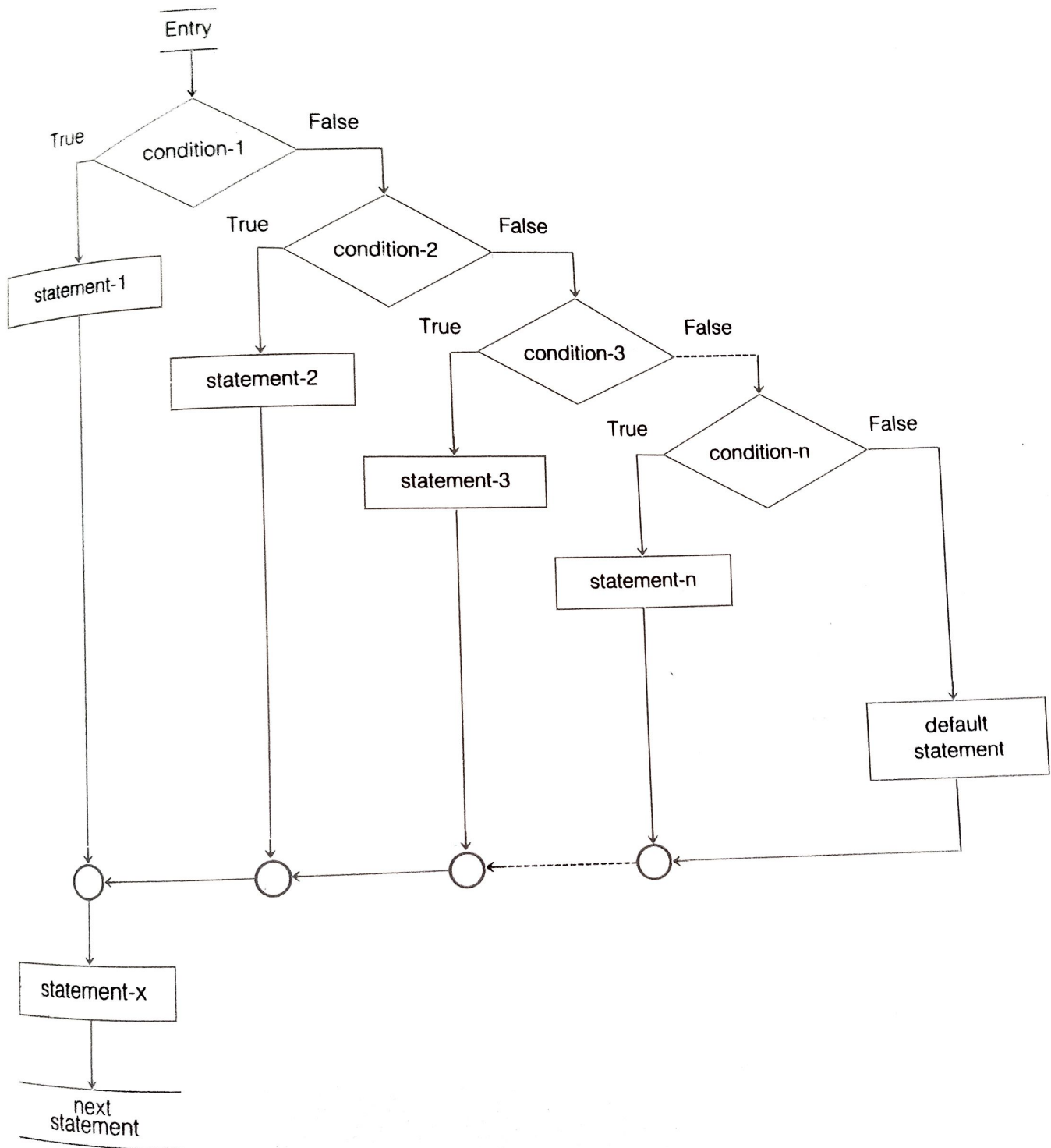


Fig. 5.9 Flowchart of **else..if** ladder

```

-----
-----
if (code == 1)
    colour = "RED";
else if (code == 2)
    colour = "GREEN";
else if (code == 3)
    colour = "WHITE";
else
    colour = "YELLOW";
-----
-----

```

Code numbers other than 1, 2 or 3 are considered to represent YELLOW colour. The same results can be obtained by using nested **if...else** statements.

```

if (code != 1)
    if (code != 2)
        if (code != 3)
            colour = "YELLOW";
        else
            colour = "WHITE";
    else
        colour = "GREEN";
else
    colour = "RED";

```

In such situations, the choice of the method is left to the programmer. However, in order to choose an **if** structure that is both effective and efficient, it is important that the programmer is fully aware of the various forms of an **if** statement and the rules governing their nesting.

Example 5.5

An electric power distribution company charges its domestic consumers as follows:

<i>Consumption Units</i>	<i>Rate of Charge</i>
0 - 200	Rs. 0.50 per unit
201 - 400	Rs. 100 plus Rs.0.65 per unit excess of 200
401 - 600	Rs. 230 plus Rs.0.80 per unit excess of 400
601 and above	Rs. 390 plus Rs.1.00 per unit excess of 600

The program in Fig.5.10 reads the customer number and power consumed and prints the amount to be paid by the customer.

5.7 THE SWITCH STATEMENT

We have seen that when one of the many alternatives is to be selected, we can design a program using **if** statements to control the selection. However, the complexity of such a

Program

```

/*.....*/
/*      USE OF else if LADDER      */
/*.....*/
main( )
{
    int units, custnum;
    float charges;

    printf("Enter CUSTOMER NO. and UNITS consumed\n");
    scanf("%d %d", &custnum, &units);
    if (units <= 200)
        charges = 0.5 * units;
    else if (units <= 400)
        charges = 100 + 0.65 * (units - 200);
    else if (units <= 600)
        charges = 230 + 0.8 * (units - 400);
    else
        charges = 390 + (units - 600);
    printf("\n\nCustomer No: %d: Charges = %.2f\n",
        custnum, charges);
}

```

Output

```

Enter CUSTOMER NO. and UNITS consumed 101 150
Customer No:101 Charges = 75.00
Enter CUSTOMER NO. and UNITS consumed 202 225
Customer No:202 Charges = 116.25
Enter CUSTOMER NO. and UNITS consumed 303 375
Customer No:303 Charges = 213.75
Enter CUSTOMER NO. and UNITS consumed 404 520
Customer No:404 Charges = 326.00
Enter CUSTOMER NO. and UNITS consumed 505 625
Customer No:505 Charges = 415.00

```

Fig. 5.10 Illustration of **else..if** ladder

program increases dramatically when the number of alternatives increases. The program becomes difficult to read and follow. At times, it may confuse even the person who designed it. Fortunately, C has a built-in multiway decision statement known as a **switch**. The **switch** statement tests the value of a given variable (or expression) against a list of **case** values and when a match is found, a block of statements associated with that **case** is executed. The general form of the **switch** statement is as shown below:

The *expression* is an integer expression or characters. *Value-1, value-2* are constants or constant expressions (evaluable to an integral constant) and are known as *case labels*. Each of these values should be unique within a **switch** statement. **block-1, block-2** are statement lists and may contain zero or more statements. There is no need to put braces around these blocks. Note that **case** labels end with a colon (:).

When the **switch** is executed, the value of the expression is successively compared against the values *value-1, value-2,* If a case is found whose value matches with the value of the expression, then the block of statements that follows the case are executed.

```

switch (expression)
{
    case value-1:
        block-1
        break;
    case value-2:
        block-2
        break;
    .....
    .....
    default:
        default-block
        break;
}
statement-x;

```

The **break** statement at the end of each block signals the end of a particular case and causes an exit from the **switch** statement, transferring the control to the **statement-x** following the **switch**.

The **default** is an optional case. When present, it will be executed if the value of the *expression* does not match with any of the case values. If not present, no action takes place if all matches fail and the control goes to the **statement-x**.

The selection process of **switch** statement is illustrated in the flowchart shown in Fig. 5.11.

The **switch** statement can be used to grade the students as discussed in the last section. This is illustrated below:

```

-----
-----
index = marks/10;
switch (index)
{
    case 10:
    case 9:
    case 8:
        grade = "Honours";
        break;
    case 7:
    case 6:
        grade = "First Division";
        break;
    case 5:
        grade = "Second Division";
        break;
    case 4:
        grade = "Third Division";
        break;
    default:
        grade = "Fail";
        break;
}
printf("%s\n", grade);
-----
-----

```

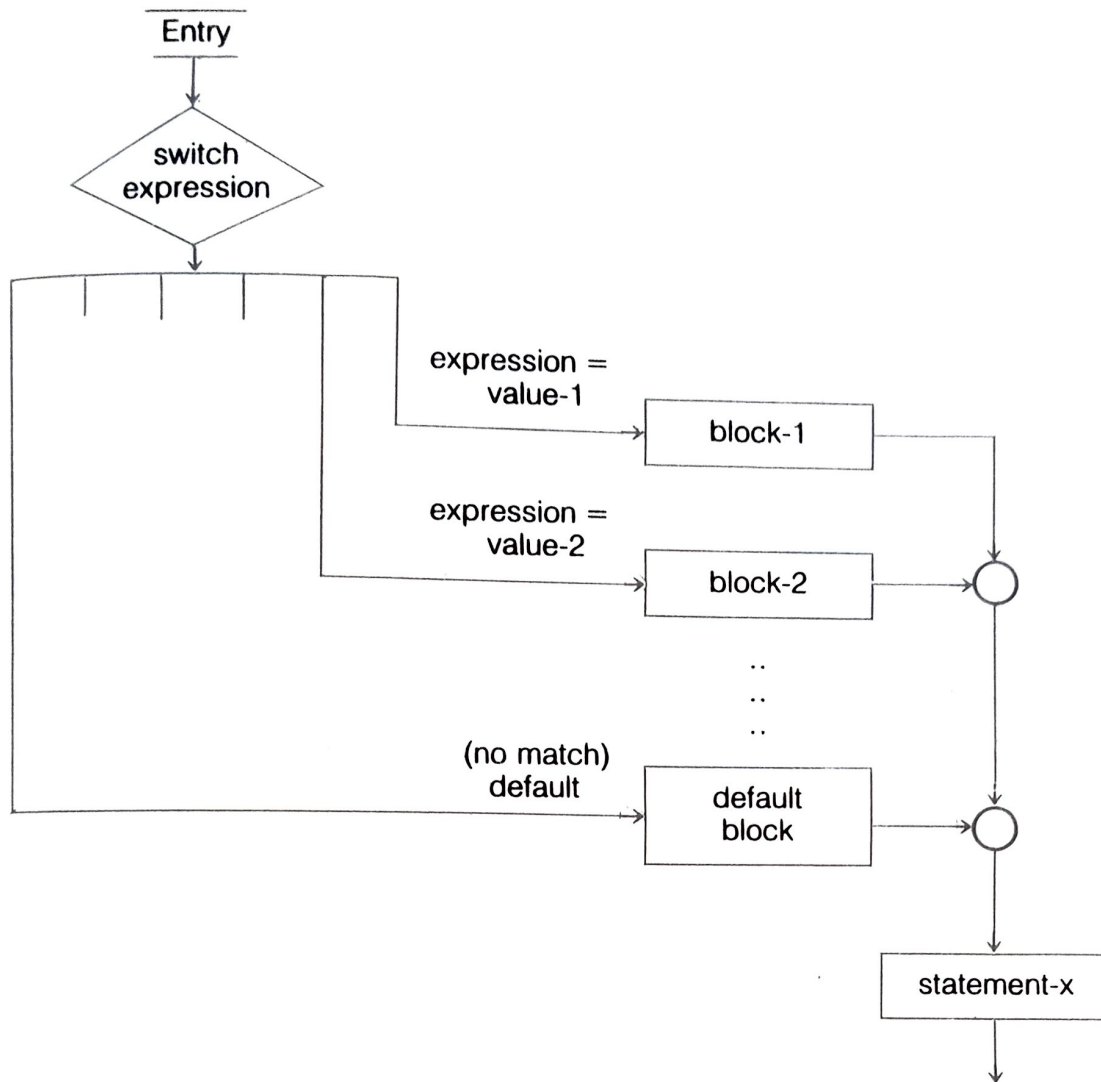


Fig. 5.11 Selection process of the **switch** statement

Note that we have used a conversion statement

index = marks / 10;

where, **index** is defined as an integer. The variable **index** takes the following integer values.

Marks	Index
100	10
90 – 99	9
80 – 89	8
70 – 79	7
60 – 69	6
50 – 59	5
40 – 49	4
.	.
0	0

This segment of the program illustrates two important features. First, it uses empty cases. The first three cases will execute the same statements

grade = "Honours";
break;

Same is the case with case 7 and case 6. Second, **default** condition is used for all other cases where marks is less than 40.

The **switch** statement is often used for menu selection. For example:

```

-----
printf(" TRAVEL GUIDE\n\n");
printf(" A Air Timings\n" );
printf(" T Train Timings\n");
printf(" B Bus Service\n" );
printf(" X To skip\n" );
printf("\n Enter your choice\n");
character = getchar( );
switch (character)
{
    case 'A' :
        air-display( );
        break;
    case 'B' :
        bus-display( );
        break;
    case 'T' :
        train-display( );
        break;
    default :
        printf(" No choice\n");
}
-----

```

It is possible to nest the **switch** statements, That is, a **switch** may be part of a **case** block. ANSI C allows 15 levels of nesting and as many as 257 *case labels*.

5.8 THE ? : OPERATOR

The C language has an unusual operator, useful for making two-way decisions. This operator is a combination of ? and :, and takes three operands. This operator is popularly known as the *conditional operator*. The general form of use of the conditional operator is as follows:

conditional expression ? *expression1* : *expression2*

The *conditional expression* is evaluated first. If the result is nonzero, *expression1* is evaluated and is returned as the value of the conditional expression. Otherwise, *expression2* is evaluated and its value is returned. For example, the segment

```

if (x < 0)
    flag = 0;
else
    flag = 1;

```

can be written as

```

flag = ( x < 0 ) ? 0 : 1;

```

Consider the evaluation of the following function:

$$\begin{aligned} y &= 1.5x + 3 && \text{for } x \leq 2 \\ y &= 2x + 5 && \text{for } x > 2 \end{aligned}$$

This can be evaluated using the conditional operator as follows:

$$y = (x > 2) ? (2 * x + 5) : (1.5 * x + 3);$$

The conditional operator may be nested for evaluating more complex assignment decisions. For example, consider the weekly salary of a salesgirl who is selling some domestic products. If x is the number of products sold in a week, her weekly salary is given by

$$\text{salary} = \begin{cases} 4x + 100 & \text{for } x < 40 \\ 300 & \text{for } x = 40 \\ 4.5x + 150 & \text{for } x > 40 \end{cases}$$

This complex equation can be written as

$$\text{salary} = (x \neq 40) ? ((x < 40) ? (4 * x + 100) : (4.5 * x + 150)) : 300;$$

The same can be evaluated using **if...else** statements as follows:

```
if (x <= 40)
    if (x < 40)
        salary = 4 * x + 100;
    else
        salary = 300;
else
    salary = 4.5 * x + 150;
```

When the conditional operator is used, the code becomes more concise and perhaps, more efficient. However, the readability is poor. It is better to use **if** statements when more than a single nesting of conditional operator is required.

Example 5.6

An employee can apply for a loan at the beginning of every six months, but he will be sanctioned the amount according to the following company rules:

Rule 1: An employee cannot enjoy more than two loans at any point of time.

Rule 2: Maximum permissible total loan is limited and depends upon the category of the employee.

A program to process loan applications and to sanction loans is given in Fig. 5.12.

The program uses the following variables:

loan3 - present loan amount requested
 loan2 - previous loan amount pending
 loan1 - previous to previous loan pending
 maxloan - maximum permissible loan
 sancloan - loan sanctioned

The rules for sanctioning new loan are:

1. loan1 should be zero.
 2. loan2 + loan3 should not be more than maxloan.
- Note the use of **long int** type to declare variables.

Program

```

/*****
      CONDITIONAL OPERATOR
*****/

#define    MAXLOAN    50000

main( )
{
    long int loan1, loan2, loan3, sancloan, sum23;
    printf("Enter the values of previous two loans:\n");
    scanf(" %ld %ld", &loan1, &loan2);
    printf("\nEnter the value of new loan:\n");
    scanf(" %ld", &loan3);
    sum23 = loan2 + loan3;
    sancloan = (loan1 > 0) ? 0 : ( (sum23 > MAXLOAN) ?
                                MAXLOAN - loan2 : loan3);

    printf("\n\n");
    printf("Previous loans pending:\n%ld %ld\n", loan1, loan2);
    printf("Loan requested = %ld\n", loan3);
    printf("Loan sanctioned = %ld\n", sancloan);
}

```

Output

Enter the values of previous two loans:
0 20000

Enter the value of new loan:
45000

Previous loans pending:
0 20000
Loan requested = 45000
Loan sanctioned = 30000

Enter the values of previous two loans:
1000 15000

Enter the value of new loan:
25000

Previous loans pending:
1000 15000
Loan requested = 25000
Loan sanctioned = 0

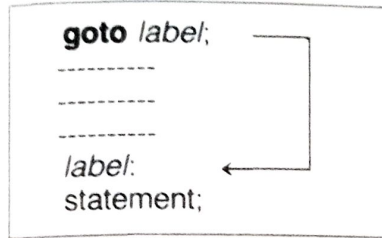
Fig. 5.12 *Illustration of the conditional operator*

5.9 THE GOTO STATEMENT

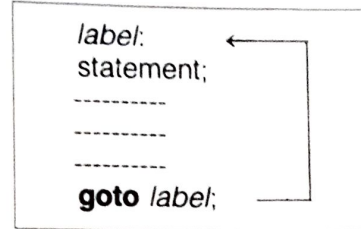
So far we have discussed ways of controlling the flow of execution based on certain specified conditions. Like many other languages, C supports the **goto** statement to branch unconditionally from one point to another in the program. Although it may not be essential to use the

goto statement in a highly structured language like C, there may be occasions when the use of *goto* might be desirable.

The *goto* requires a *label* in order to identify the place where the branch is to be made. A *label* is any valid variable name, and must be followed by a colon. The *label* is placed immediately before the statement where the control is to be transferred. The general forms of *goto* and *label* statements are shown below:



Forward jump



Backward jump

The *label:* can be anywhere in the program either before or after the *goto label; statement*. During running of a program when a statement like

goto begin;

is met, the flow of control will jump to the statement immediately following the label **begin:**. This happens unconditionally.

Note that a *goto* breaks the normal sequential execution of the program. If the *label:* is before the statement *goto label;* a *loop* will be formed and some statements will be executed repeatedly. Such a jump is known as a *backward jump*. On the other hand, if the *label:* is placed after the *goto label;* some statements will be skipped and the jump is known as a *forward jump*.

A *goto* is often used at the end of a program to direct the control to go to the input statement, to read further data. Consider the following example:

```
main( )
{
    double x, y;
    read:
    scanf("%f", &x);
    if (x < 0) goto read;
    y = sqrt(x);
    printf("%f %f\n", x, y);
    goto read;
}
```

This program is written to evaluate the square root of a series of numbers read from the terminal. The program uses two *goto* statements, one at the end, after printing the results to transfer the control back to the input statement and the other to skip any further computation when the number is negative.

Due to the unconditional *goto* statement at the end, the control is always transferred back to the input statement. In fact, this program puts the computer in a permanent loop known as an *infinite loop*. The computer goes round and round until we take some special steps to terminate the loop. Such infinite loops should be avoided. Example 5.7 illustrates how such infinite loops can be eliminated.

Example 5.7

Program presented in Fig. 5.13 illustrates the use of the **goto** statement.

The program evaluates one square root for five numbers. The variable **count** keeps the count of numbers read. When **count** is less than or equal to 5, **goto read**; directs the control to the label **read**; otherwise, the program prints a message and stops.

```

Program
/*****
 *
 * USE OF goto STATEMENT
 *
 *****/
#include <math.h>

main()
{
    double x, y;
    int count;
    count = 1;

    read:
        printf("Enter FIVE real values in a LINE \n");
        scanf("%lf", &x);
        printf("\n");
        if (x < 0)
            printf("Item - %d is negative\n", count);
        else
        {
            y = sqrt(x);
            printf("%f\t%f\n", x, y);
        }

        count = count + 1;
        if (count <= 5)
            goto read;
        printf("\nEnd of computation");
    }

Output

Enter FIVE real values in a LINE
50.70 40 -36 75 11.25
50.750000 7.123903
40.000000 6.324555
Item -3 is negative
75.000000 8.660254
11.250000 3.354102

End of computation

```

Fig. 5.13 Use of the **goto** statement

Another use of the **goto** statement is to transfer the control out of a loop (or nested loops) when certain peculiar conditions are encountered. Example:

```

-----
while(-----)
{
    for(-----)
    {
        -----
        if(-----) goto end_of_program;
        -----
    }
}
end_of_program:

```

Jumping out
of loops

We should try to avoid using **goto** as far as possible. But there is nothing wrong, if we use it to enhance the readability of the program or to improve the execution speed.

CASE STUDIES

1. Range of Numbers

Problem: A survey of the computer market shows that personal computers are sold at varying costs by the vendors. The following is the list of costs (in hundreds) quoted by some vendors:

35.00	40.50	25.00	31.25	68.15
47.00	26.65	29.00	53.45	62.50

Determine the average cost and the range of values.

Problem analysis: Range is one of the measures of dispersion used in statistical analysis of a series of values. The range of any series is the difference between the highest and the lowest values in the series. That is

$$\text{Range} = \text{highest value} - \text{lowest value}$$

It is therefore necessary to find the highest and the lowest values in the series.

Program: A program to determine the range of values and the average cost of a personal computer in the market is given in Fig. 5.14.

When the value is read the first time, it is assigned to two *buckets*, **high** and **low**, through the statement

high = low = value;

For subsequent values, the value read is compared with **high**; if it is larger, the value is assigned to **high**. Otherwise, the value is compared with **low**; if it is smaller, the value is assigned to **low**. Note that at a given point, the buckets **high** and **low** hold the highest and the lowest values read so far.

```

Program
/*****
/*
RANGE OF NUMBERS
*****/
main()
{
    int count;
    float value, high, low, sum, average, range;
    sum = 0;
    count = 0;
    printf("Enter numbers in a line :
input a NEGATIVE number to end\n");
    input:
    scanf("%f", &value);
    if (value < 0) goto output;
    count = count + 1;
    if (count == 1)
        high = low = value;
    else if (value > high)
        high = value;
    else if (value < low)
        low = value;
    sum = sum + value;
    goto input;
output:
    average = sum/count;
    range = high - low;
    printf("Total values : %d\n", count);
    printf("Highest-value: %f\nLowest-value : %f\n", high, low);
    printf("Range      : %f\nAverage      : %f\n",
range, average);
}

Output
Enter numbers in a line : input a NEGATIVE number to end
35 40.50 25 31.25 68.15 47 26.65 29 53.45 62.50 -1

Total values : 10
Highest-value : 68.150002
Lowest-value : 25.000000
Range : 43.150002
Average : 41.849998

```

Fig. 5.14 Calculation of range of values

The values are read in an input loop created by the **goto input**; statement. The control is transferred out of the loop by inputting a negative number. This is caused by the statement

If (value < 0) goto output;

2. Pay-Bill Calculations

Problem: A manufacturing company has classified its executives into four levels for the benefit of certain perks. The levels and corresponding perks are shown below:

Level	Perks	
	Conveyance allowance	Entertainment allowance
1	1000	500
2	750	200
3	500	100
4	250	

An executive's gross salary includes basic pay, house rent allowance at 25% of basic pay and other perks. Income tax is withheld from the salary on a percentage basis as follows:

Gross salary	Tax rate
Gross $\leq$ 2000	No tax deduction
2000 < Gross $\leq$ 4000	3%
4000 < Gross $\leq$ 5000	5%
Gross > 5000	8%

Write a program that will read an executive's job number, level number, and basic pay and then compute the net salary after withholding income tax.

Problem analysis:

Gross salary = basic pay + house rent allowance + perks

Net salary = gross salary - income tax.

The computation of perks depends on the level, while the income tax depends on the gross salary. The major steps are:

1. Read data.
2. Decide level number and calculate perks.
3. Calculate gross salary.
4. Calculate income tax.
5. Compute net salary.
6. Print the results.

Program: A program and the results of the test data are given in Fig. 5.15. Note that the last statement should be an executable statement. That is, the label stop; cannot be the last line.

```

Program
/*****
*/
/*          PAY-BILL CALCULATIONS          */
/*****
/

#define CA1      1000
#define CA2      750
#define CA3      500
#define CA4      250
#define EA1      500
#define EA2      200
#define EA3      100
#define EA4       0

main()
{
    int  level, jobnumber;
    float gross,
          basic,
          house_rent,
          perks,
          net,
          incometax;

    input:
    printf("\nEnter level, job number, and basic pay\n");
    printf("Enter 0 (zero) for level to END\n\n");
    scanf("%d", &level);
    if (level == 0) goto stop;
    scanf("%d %d", &jobnumber, &basic);
    switch (level)
    {
        case 1:
            perks = CA1 + EA1;
            break;
        case 2:
            perks = CA2 + EA2;
            break;
        case 3:
            perks = CA3 + EA3;
            break;
        case 4:
            perks = CA4 + EA4;
            break;
        default:
            printf("Error in level code\n\n");
            goto stop;
    }
    house_rent = 0.25 * basic;
    gross = basic + house_rent + perks;
    if (gross <= 2000)
        incometax = 0;
    else if (gross <= 4000)
        incometax = 0.03 * gross;

```

```

else if (gross <= 5000)
    incometax = 0.05 * gross;
else
    incometax = 0.08 * gross;
net = gross - incometax;
printf("%d %d %.2f\n", level, jobnumber, net);
goto input;
stop: printf("\n\nEND OF THE PROGRAM");
}

```

Output

Enter level, job number, and basic pay
Enter 0 (zero) for level to END

1 1111 4000
1 1111 5980.00

Enter level, job number, and basic pay
Enter 0 (zero) for level to END

2 2222 3000
2 2222 4465.00

Enter level, job number, and basic pay
Enter 0 (zero) for level to END

3 3333 2000
3 3333 3007.00

Enter level, job number, and basic pay
Enter 0 (zero) for level to END

4 4444 1000
4 4444 1500.00

Enter level, job number, and basic pay
Enter 0 (zero) for level to END

0

END OF THE PROGRAM

Fig. 5.15 *Pay-bill calculations*

REVIEW QUESTIONS AND EXERCISES

- 5.1 Determine whether the following are true or false:
 - (a) When if statements are nested, the last **else** gets associated with the nearest if without an **else**.
 - (b) One if can have more than one **else** clause.
 - (c) A **switch** statement can always be replaced by a series of **if..else** statements.
 - (d) A **switch** expression can be of any type.
 - (e) A program stops its execution when a **break** statement is encountered.
- 5.2 In what ways does a **switch** statement differ from an **if** statement?
- 5.3 Find errors, if any, in each of the following segments:

- (a) if ($x + y = z$ && $y > 0$)
 printf(" ");
- (b) if (code > 1);
 a = b + c
 else
 a = 0
- (c) if ($p < 0$) || ($q < 0$)
 printf(" sign is negative");

5.4 The following is a segment of a program:

```

x = 1;
y = 1;
if (n > 0)
    x = x + 1;
    y = y - 1;
printf(" %d %d", x, y);

```

What will be the values of x and y if n assumes a value of (a) 1 and (b) 0.

5.5 Rewrite each of the following without using compound relations:

- (a) if (grade <= 59 && grade >= 50)
 second = second + 1;
- (b) if (number > 100 || number < 0)
 printf(" Out of range");
 else
 sum = sum + number;
- (c) if ((M1 > 60 && M2 > 60) || T > 200)
 printf(" Admitted\n");
 else
 printf(" Not admitted\n");

5.6 Read all the programs discussed in this chapter. Identify any changes that might be necessary to improve either readability or efficiency of the programs.

5.7 Write a program to determine whether a number is 'odd' or 'even' and print the message

NUMBER IS EVEN

or

NUMBER IS ODD

(a) without using else option, and (b) with else option.

5.8 Write a program to find the number of and sum of all integers greater than 100 and less than 200 that are divisible by 7.

5.9 A set of two linear equations with two unknowns x_1 and x_2 is given below:

$$ax_1 + bx_2 = m$$

$$cx_1 + dx_2 = n$$

The set has a unique solution

$$x_1 = \frac{md - bn}{ad - cb}$$

$$x_2 = \frac{na - mc}{ad - cb}$$

provided the denominator $ad - cb$ is not equal to zero.

Write a program that will read the values of constants a, b, c, d, m , and n and compute the values of x_1 and x_2 . An appropriate message should be printed if $ad - cb = 0$.

- 5.10 Given a list of marks ranging from 0 to 100, write a program to compute and print the number of students:

- (a) who have obtained more than 80 marks,
- (b) who have obtained more than 60 marks,
- (c) who have obtained more than 40 marks,
- (d) who have obtained 40 or less marks,
- (e) in the range 81 to 100,
- (f) in the range 61 to 80,
- (g) in the range 41 to 60, and
- (h) in the range 0 to 40.

The program should use a minimum number of **if** statements.

- 5.11 Admission to a professional course is subject to the following conditions:

- (a) Marks in mathematics ≥ 60
- (b) Marks in physics ≥ 50
- (c) Marks in Chemistry ≥ 40
- (d) Total in all three subjects ≥ 200

or

Total in mathematics and physics ≥ 150

Given the marks in the three subjects, write a program to process the applications to list the eligible candidates.

- 5.12 Write a program to print a two-dimensional Square Root Table as shown below, to provide the square root of any number from 0 to 9.9. For example, the value x will give the square root of 3.2 and y the square root of 3.9.

Square Root Table

Number	0.0	0.1	0.2		0.9
0.0					
1.0					
2.0					
3.0			x		y
9.0					

5.13 Shown below is a Floyd's triangle.

```

1
2 3
4 5 6
7 8 9 10
11 . . . . . 15
.
.
.
79 . . . . . 91

```

- (a) Write a program to print this triangle.
 (b) Modify the program to produce the following form of Floyd's triangle.

```

1
0 1
1 0 1
0 1 0 1
1 0 1 0 1

```

5.14 A cloth showroom has announced the following seasonal discounts on purchase of items:

Purchase amount	Discount	
	Mill cloth	Handloom items
0 - 100	—	5%
101 - 200	5%	7.5%
201 - 300	7.5%	10.0%
Above 300	10.0%	15.0%

Write a program using **switch** and **if** statements to compute the net amount to be paid by a customer.

5.15 Write a program that will read the value of x and evaluate the following function

$$y = \begin{cases} 1 & \text{for } x > 0 \\ 0 & \text{for } x = 0 \\ -1 & \text{for } x < 0 \end{cases}$$

Using

- (a) nested **if** statements,
 (b) **else if** statements, and
 (c) conditional operator ? :